

CN-Market Stock Market Plugin

Technical Documentation & Developer Guide

CraftNepal Development Team

July 5, 2026

Contents

1	Introduction	2
2	System Architecture	3
2.1	Component Overview	3
2.2	Data Flow	3
2.3	Database Schema	4
3	Stock Market Engine	5
3.1	File Location	5
3.2	Order Types	5
3.3	Order Matching Algorithm	6
3.3.1	Price Discovery	6
3.4	Stop-Order Evaluation	6
3.5	Portfolio Management	7
4	Authentication System	8
4.1	API Key Lifecycle	8
4.1.1	Generation	8
4.1.2	Expiration and Renewal	8
4.1.3	Revocation	8
4.2	Two-Factor Authentication (TOTP)	8
4.3	Session Management	9
4.4	Lockout Policy	9
4.5	Rate Limiting	9
5	REST API	11
5.1	Server Details	11
5.2	Authentication Endpoints	11
5.3	Market Data Endpoints	11
5.4	Trading Endpoints	12
5.5	CORS and Error Handling	13
6	WebSocket Protocol	14
6.1	Server Details	14
6.2	Connection	14
6.3	Subscription Protocol	14
6.4	Broadcast Channels	15
6.5	Message Formats	15
6.6	Keepalive	16

7	OHLCV Aggregation Engine	17
7.1	Purpose	17
7.2	Intervals	17
7.3	Aggregation Algorithm	17
7.4	Query	18
8	Admin Commands	19
8.1	Pause	19
8.2	Resume	19
8.3	Inject Test Trade	20
8.4	Status	20
8.5	Complete Subcommand Registration	21
9	Frontend Application	22
9.1	Technology Stack	22
9.2	Project Structure	22
9.3	Key Components	23
9.3.1	WebSocket Client (<code>api/websocket.ts</code>)	23
9.3.2	Dashboard Layout	23
9.3.3	Candlestick Chart (<code>components/Chart.tsx</code>)	24
9.3.4	Order Form (<code>components/OrderForm.tsx</code>)	24
9.4	Auth Flow	24
10	Configuration	25
10.1	Port Configuration	25
10.2	Rate Limiting	25
11	Building and Deployment	26
11.1	Prerequisites	26
11.2	Building the Plugin	26
11.3	Running the Frontend	26
11.4	Production Build	26
11.5	Production Deployment (nginx)	27
11.6	Docker Test Server	27
11.7	Testing with Injected Trades	28
12	Complete Source Listings	29
12.1	Key File Reference	29
12.2	Total Size	29

Chapter 1

Introduction

The CN-Market plugin is a Spigot/Paper 1.21.x Minecraft server plugin that provides a persistent market world with player-owned shops, admin shops, currency exchange, and—since version 1.1.4—a full real-time stock market system with web-based trading.

This document covers the stock market subsystem: its architecture, authentication model, REST and WebSocket APIs, order matching engine, candle aggregation, and the companion React frontend.

The stock market allows players to trade virtual “shares” backed by in-game item tickers (e.g. DIAMOND, IRON_INGOT). All trades settle against the player’s Vault economy balance in real time. No physical items move; the system is a pure speculation layer on top of the existing shop economy.

Chapter 2

System Architecture

2.1 Component Overview

The stock market system consists of the following components:

StockMarketEngine Core singleton that maintains the in-memory order book, executes trades, manages portfolios, and persists all state to SQLite.

EmbeddedServer JDK `com.sun.net.httpserver.HttpServer` running on a configurable port (default 8080). Serves REST JSON endpoints for authentication, market data, and trading.

WebSocketManager Separate server (default port 8081) that implements the RFC 6455 WebSocket protocol. Broadcasts real-time ticker snapshots, trade executions, and order book updates.

ApiKeyManager Handles API key generation (PBKDF2WithHmacSHA256, 120k iterations), session tokens, TOTP setup, logout logic, and backup code verification.

TwoFactorAuth RFC 6238 TOTP implementation using HMAC-SHA1 and Base32-encoded secrets.

CandleAggregator Background task that builds OHLCV candlestick data from the trade history at five interval granularities.

RateLimiter Sliding-window rate limiter applied to all REST and WebSocket traffic.

transform.ts Shared TypeScript utility that normalizes all `snake_case` API and WebSocket responses to `camelCase`, providing a consistent interface to all frontend components.

2.2 Data Flow

1. A player generates an API key in-game: `/market apikey generate`. The plaintext key is shown once, its PBKDF2 hash is stored in SQLite.
2. The player visits the web frontend and enters the API key on the login page.

3. The frontend calls `POST /api/v1/auth/step1` with the key. The server resolves the key to a UUID, checks expiration and logout status, and returns the UUID plus TOTP setup data if first-time setup is needed.
4. The frontend calls `POST /api/v1/auth/step2` with the UUID and a TOTP code from the player's authenticator app. On success, a session cookie (`market_session`) is set and a session token is returned.
5. All subsequent API calls carry the session cookie via `credentials: 'include'`. The server resolves the session to a UUID and executes the requested operation (place order, cancel order, query portfolio, etc.).
6. All API and WebSocket responses use `snake_case` field names. The frontend's `transformKeys()` utility normalizes them to `camelCase` before they reach React components.
7. When a trade executes, `StockMarketEngine` calls `broadcastTrades()`, which pushes the trade event to `WebSocketManager`. The WebSocket server broadcasts to all subscribed clients.
8. A separate tick loop runs every 60 seconds: evaluate stop-loss and take-profit orders, aggregate OHLCV candles, clean up old data.

2.3 Database Schema

All persistent state lives in a single SQLite file at `plugins/Market/market.db`. The `stock-market` tables are:

Table	Purpose	Key Columns
<code>api_keys</code>	API key hashes, TOTP, sessions	<code>uuid</code> , <code>key_hash</code> , <code>totp_secret</code> , <code>session_token</code>
<code>stock_orders</code>	All orders (open/filled/cancelled)	<code>id</code> , <code>ticker</code> , <code>type</code> , <code>side</code> , <code>price</code> , <code>quantity</code> , <code>status</code>
<code>stock_trades</code>	Executed trade log	<code>id</code> , <code>ticker</code> , <code>price</code> , <code>quantity</code> , <code>buyer</code> , <code>seller</code> , <code>timestamp</code>
<code>stock_portfolios</code>	Player share holdings	<code>uuid</code> , <code>ticker</code> , <code>shares</code> , <code>avg_entry_price</code>
<code>stock_candles</code>	OHLCV aggregated candles	<code>ticker</code> , <code>interval</code> , <code>open_time</code> , <code>o/h/l/c</code> , <code>volume</code>

Chapter 3

Stock Market Engine

3.1 File Location

All stock market engine code resides in:

```
src/main/java/net/craftnepal/market/stock/
```

The main engine is `StockMarketEngine.java` (~990 lines).

3.2 Order Types

The engine supports four order types, defined in the `OrderType` enum at line 24:

```
1 public enum OrderType {  
2     MARKET, LIMIT, STOP_LOSS, TAKE_PROFIT  
3 }
```

Listing 3.1: Order types (StockMarketEngine.java:24)

MARKET Executes immediately against the best available opposite orders in the book. No price limit is required.

LIMIT Executes only at the specified price or better. If not fully filled, the remainder sits in the order book.

STOP_LOSS A sell order that sits in the book until the market price drops to or below the stop price. When triggered, it converts to a **MARKET** sell and executes immediately.

TAKE_PROFIT A sell order that sits in the book until the market price rises to or above the target price. When triggered, it converts to a **MARKET** sell.

Price clamping is enforced for **LIMIT** orders at lines 224–229:

```
1 if (type == OrderType.LIMIT && price > 0) {  
2     double maxAllowed = referencePrice * 1.20;  
3     double minAllowed = referencePrice * 0.80;  
4     if (price > maxAllowed) price = maxAllowed;  
5     if (price < minAllowed) price = minAllowed;  
6 }
```

Listing 3.2: Price clamping (StockMarketEngine.java:224-229)

3.3 Order Matching Algorithm

The engine uses an in-memory order book implemented with `ConcurrentHashMap<String, NavigableMap<Double, List<Order>>>` (lines 126–127):

```
1 private static final ConcurrentHashMap<String, NavigableMap<
    Double, List<Order>>>
2     bidsByTicker = new ConcurrentHashMap<>();
3 private static final ConcurrentHashMap<String, NavigableMap<
    Double, List<Order>>>
4     asksByTicker = new ConcurrentHashMap<>();
```

Listing 3.3: Order book data structures (StockMarketEngine.java:126-127)

Bids are stored in descending order (highest bid first), asks in ascending order (lowest ask first). The matching logic for a buy market order (`matchBuyMarket`, line 325) iterates ask levels from low to high, matching as much quantity as possible at each price level. For limit orders (`matchBid`, line 428), matching stops when the ask price exceeds the bid price.

3.3.1 Price Discovery

When a trade executes, the trade price is recorded as the last price for that ticker (line 514):

```
1 // Update last price
2 lastPrices.put(buyOrder.ticker, tradePrice);
```

Listing 3.4: Last price update (StockMarketEngine.java:513-514)

If no trades exist for a ticker, the engine falls back to `DynamicPriceManager.getDynamicPrice()` which provides a fundamental value based on the item's shop price history.

3.4 Stop-Order Evaluation

The `evaluateStopOrders()` method (line 777) runs on a 60-second async timer from `Market.java`. It queries all OPEN stop-loss and take-profit orders from the database, checks the current market price against each order's stop price, and triggers matching orders as MARKET orders:

```
1 boolean shouldTrigger = false;
2 if (order.type == OrderType.STOP_LOSS) {
3     if (order.side == OrderSide.SELL && currentPrice <= order.
        stopPrice) {
4         shouldTrigger = true;
5     }
6 } else if (order.type == OrderType.TAKE_PROFIT) {
7     if (order.side == OrderSide.SELL && currentPrice >= order.
        stopPrice) {
8         shouldTrigger = true;
9     }
10 }
```

Listing 3.5: Stop-order trigger logic (StockMarketEngine.java:795-803)

3.5 Portfolio Management

Player share holdings are tracked in two maps (lines 128–129):

```
1 private static final ConcurrentHashMap<String, HashMap<String, Integer>>
   playerShares = new ConcurrentHashMap<>();
3 private static final ConcurrentHashMap<String, HashMap<String, Double>>
   playerAvgPrice = new ConcurrentHashMap<>();
```

Listing 3.6: Portfolio state (StockMarketEngine.java:128-129)

The `getPortfolio()` method (line 621) computes per-holding P&L and returns a structured map with total value, cost, and percentage returns.

Chapter 4

Authentication System

4.1 API Key Lifecycle

4.1.1 Generation

Keys are generated by `ApiKeyManager.generateKey()`:

```
1 public static String[] generateKey(String uuid) {
2     byte[] keyBytes = new byte[32];
3     RANDOM.nextBytes(keyBytes);
4     StringBuilder key = new StringBuilder("mk_");
5     for (byte b : keyBytes) key.append(String.format("%02x", b
6         & 0xff));
7     String plainKey = key.toString();
8     String hash = hashKey(plainKey);
9     // store hash in DB, return plaintext key
10 }
```

Listing 4.1: API key generation (`ApiKeyManager.java`)

The plaintext key (format: `mk_` followed by 64 hex characters) is displayed to the player exactly once via the in-game command and is never persisted. The hash uses PBKDF2WithHmacSHA256 with 120,000 iterations.

4.1.2 Expiration and Renewal

Keys expire after 7 days. The player can renew via `/market apikey renew` before expiration. Expired keys return 401 from the API.

4.1.3 Revocation

`/market apikey revoke` immediately deletes the key hash from the database, invalidating all active sessions.

4.2 Two-Factor Authentication (TOTP)

When a player first authenticates through the website, the server checks whether TOTP has been set up. If not, it generates a secret and returns a provisioning URL for the

authenticator app:

```
1 if (!data.totpEnabled) {
2     String totpSecret = TwoFactorAuth.generateSecret();
3     String[] backupCodes = TwoFactorAuth.generateBackupCodes();
4     String qrUrl = TwoFactorAuth.getQRCodeURL(getPlayerName(
5         uuid), totpSecret);
6     resp.put("totp_setup_needed", true);
7     resp.put("totp_secret", totpSecret);
8     resp.put("qr_code_url", qrUrl);
9     resp.put("backup_codes", String.join(", ", backupCodes));
10    ApiKeyManager.setupTOTP(uuid, totpSecret, backupCodes);
11 }
```

Listing 4.2: TOTP setup response (EmbeddedServer.java:127-135)

The TOTP implementation follows RFC 6238 using HMAC-SHA1 and 30-second time steps. Backup codes are hashed with the same PBKDF2 function and stored as a JSON array in the `backup_codes` column.

4.3 Session Management

After step 2, a session token is created (line 178):

```
1 String sessionToken = ApiKeyManager.createSession(uuid);
2 Map<String, Object> resp = new HashMap<>();
3 resp.put("session_token", sessionToken);
4 resp.put("expires_at", System.currentTimeMillis() + (24L * 60L
5     * 60L * 1000L));
6 setSessionCookie(exchange, sessionToken, 86400);
```

Listing 4.3: Session creation (EmbeddedServer.java:178-183)

Sessions are stored in the `api_keys` table as `session_token` and `session_expires_at`. A new login overwrites the previous session, providing single-concurrent-session semantics.

4.4 Lockout Policy

Five consecutive failed TOTP attempts triggers a 15-minute lockout (`locked_until` column). Ten failures permanently revokes the key. Attempts are tracked per UUID in the `failed_attempts` column.

4.5 Rate Limiting

The `RateLimiter` class uses a sliding-window algorithm:

```
1 public static boolean allow(String key, int maxRequests, long
2     windowMs) {
3     long now = System.currentTimeMillis();
4     ArrayDeque<Long> log = logs.computeIfAbsent(key, k -> new
5         ArrayDeque<>());
```

```
4     synchronized (log) {
5         while (!log.isEmpty() && log.peekFirst() < now -
windowMs)
6             log.pollFirst();
7         if (log.size() >= maxRequests) return false;
8         log.addLast(now);
9         return true;
10    }
11 }
```

Listing 4.4: Rate limiter core (RateLimiter.java)

Default configuration: 120 requests per 60-second window per IP/UUID. WebSocket clients are limited to 30 messages per 10 seconds.

Chapter 5

REST API

5.1 Server Details

The REST server is implemented by `EmbeddedServer.java` using JDK's built-in `com.sun.net.httpserver`. It listens on port 8080 by default (configurable in `config.yml`).

5.2 Authentication Endpoints

Method	Path	Description
POST	<code>/api/v1/auth/step1</code>	Submit API key; returns session token. Body: <code>{key: "mk_..."}</code> .
POST	<code>/api/v1/auth/step2</code>	Submit TOTP code. Body: <code>{uuid, code}</code> . Returns <code>session_token</code> cookie.
POST	<code>/api/v1/auth/logout</code>	Destroy current session.
GET	<code>/api/v1/auth/status</code>	Check authentication status.

5.3 Market Data Endpoints

Method	Path	Description
GET	<code>/api/v1/tickers</code>	All ticker snapshots sorted by absolute price change.
GET	<code>/api/v1/ticker/{ticker}</code>	Single ticker detail. E.g. <code>/api/v1/ticker/DIAMOND</code> .
GET	<code>/api/v1/orderbook/{ticker}/{depth}</code>	Bid/ask levels with quantities and order counts.
GET	<code>/api/v1/trades/{ticker}/{limit}</code>	Recent trade history.
GET	<code>/api/v1/candles/{ticker}/{interval}/{limit}</code>	OHLCV candle data. Intervals: 1m, 5m, 15m, 1h, 1d.

The `TickersHandler` (line 264) constructs a JSON array of all ticker snapshots:

```

1 List<TickerSnapshot> snapshots = StockMarketEngine.
  getAllTickers();
2 List<Map<String, Object>> list = new ArrayList<>();
3 for (TickerSnapshot s : snapshots) {
4     Map<String, Object> m = new LinkedHashMap<>();
5     m.put("ticker", s.ticker);
6     m.put("last_price", s.lastPrice);
7     m.put("bid", s.bidPrice);
8     m.put("ask", s.askPrice);
9     m.put("spread", s.spread);
10    m.put("change_24h_pct", Math.round(s.change24h * 100.0) /
100.0);
11    m.put("volume_24h", s.volume24h);
12    m.put("high_24h", s.high24h);
13    m.put("low_24h", s.low24h);
14    list.add(m);
15 }

```

Listing 5.1: Tickers response builder (EmbeddedServer.java:273-291)

5.4 Trading Endpoints

Method	Path	Description
GET	/api/v1/orders	List open orders for authenticated player.
POST	/api/v1/orders	Place an order. Body: {ticker, side, type, price?, quantity}
DELETE	/api/v1/order/{id}	Cancel an open order.
GET	/api/v1/portfolio	Portfolio holdings with P&L.
GET	/api/v1/balance	Player's Vault balance.

The OrdersHandler POST path (line 508) parses the request body and delegates to StockMarketEngine.placeOrder():

```

1 Map<String, String> body = parseForm(exchange);
2 String ticker = body.get("ticker");
3 String side = body.get("side");
4 String type = body.get("type");
5 String priceStr = body.get("price");
6 String qtyStr = body.get("quantity");
7
8 OrderType orderType = OrderType.valueOf(type.toUpperCase());
9 OrderSide orderSide = OrderSide.valueOf(side.toUpperCase());
10 double price = priceStr != null ? Double.parseDouble(priceStr)
11   : 0;
12
13 int quantity = Integer.parseInt(qtyStr);
14
15 OrderResult result = StockMarketEngine.placeOrder(

```

```
uuid, ticker, orderType, orderSide, price, quantity);
```

Listing 5.2: Order placement (EmbeddedServer.java:508-543)

5.5 CORS and Error Handling

Every handler begins with `handleCors(exchange)` which adds `Access-Control-Allow-Origin: *` and handles OPTIONS preflight requests. Errors are returned as JSON with an `error` field and `success: false`.

Chapter 6

WebSocket Protocol

6.1 Server Details

`WebSocketManager.java` implements RFC 6455 on a separate port (default 8081). The implementation is self-contained—no external dependencies—using only JDK classes.

6.2 Connection

In development, the client connects directly:

```
ws://localhost:8081
```

In production, the nginx reverse proxy forwards `/ws` on the same origin to port 8081, so the client connects via:

```
wss://market.craftnepal.net/ws
```

The frontend determines the URL automatically using `location.protocol` and `location.host` in production builds.

The server performs the standard WebSocket upgrade handshake:

```
1 MessageDigest sha1 = MessageDigest.getInstance("SHA-1");
2 sha1.update(key.getBytes());
3 sha1.update(MAGIC_GUID.getBytes());
4 acceptKey = Base64.getEncoder().toString(sha1.digest());
5
6 String response = "HTTP/1.1 101 Switching Protocols\r\n" +
7     "Upgrade: websocket\r\n" +
8     "Connection: Upgrade\r\n" +
9     "Sec-WebSocket-Accept: " + acceptKey + "\r\n" +
10    "Access-Control-Allow-Origin: *\r\n" + "\r\n";
11 out.write(response.getBytes());
```

Listing 6.1: WebSocket handshake (`WebSocketManager.java`)

6.3 Subscription Protocol

After connecting, the client sends JSON messages to subscribe to channels:


```

1 {"type": "subscribe", "channel": "tickers"}
2 {"type": "subscribe", "channel": "trades.DIAMOND"}
3 {"type": "subscribe", "channel": "orderbook.DIAMOND"}

```

Listing 6.2: Client subscribe message

Unsubscribe follows the same pattern:

```

1 {"type": "unsubscribe", "channel": "trades.DIAMOND"}

```

Listing 6.3: Client unsubscribe message

6.4 Broadcast Channels

Channel	Content	Frequency
tickers	All ticker snapshots (price, change, volume, high/low)	Every 5 seconds
trades	All executed trades (id, ticker, price, qty, timestamp)	On every match
trades.{TICKER}	Per-ticker trade feed	On every match
orderbooks	All order book depth (top 10 levels)	On every trade
orderbook.{TICKER}	Per-ticker depth	On every trade

6.5 Message Formats

Ticker broadcast:

```

1 {
2   "type": "tickers",
3   "data": [
4     {"ticker": "DIAMOND", "last_price": 150.25, "bid": 149.00,
5      "ask": 151.50, "change_24h_pct": 2.34, "volume_24h": 1200,
6      "high_24h": 155.00, "low_24h": 145.50}
7   ]
8 }

```

Listing 6.4: WebSocket ticker message

Trade broadcast:

```

1 {
2   "type": "trade",
3   "data": {
4     "id": "trd_a1b2c3d4e5f6",
5     "ticker": "DIAMOND",
6     "price": 150.25,
7     "quantity": 10,
8     "buyer_uuid": "00000000-0000-0000-0000-000000000000",
9     "seller_uuid": "00000000-0000-0000-0000-000000000000",

```

```
10     "timestamp": 1700000000000
11   }
12 }
```

Listing 6.5: WebSocket trade message

Order book broadcast:

```
1 {
2   "type": "orderbook",
3   "ticker": "DIAMOND",
4   "data": {
5     "bids": [{"price":150.00,"quantity":50,"orders":3}],
6     "asks": [{"price":151.00,"quantity":30,"orders":2}]
7   }
8 }
```

Listing 6.6: WebSocket order book message

6.6 Keepalive

The server sends a WebSocket ping frame to every connected client every 30 seconds. Clients should respond with a pong. Dead connections are cleaned up automatically.

Chapter 7

OHLCV Aggregation Engine

7.1 Purpose

The `CandleAggregator` class converts raw trade data into standardized candlestick data for charting. It runs every 60 seconds via the Bukkit scheduler.

7.2 Intervals

Five interval granularities are supported:

Interval	Period	Retention
1m	60 seconds	24 hours
5m	5 minutes	7 days
15m	15 minutes	30 days
1h	1 hour	90 days
1d	1 day	Forever

7.3 Aggregation Algorithm

For each interval, the aggregator queries trades since the last candle's close time, groups them by time bucket (`timestamp - (timestamp % intervalMs)`), and computes OHLCV for each bucket:

```
1 static class CandleBuilder {
2     double open, high, low, close;
3     int volume, tradeCount;
4     boolean initialized;
5
6     void add(double price, int qty) {
7         if (!initialized) {
8             open = high = low = close = price;
9             initialized = true;
10        } else {
11            if (price > high) high = price;
12            if (price < low) low = price;
13            close = price;
14        }
15    }
16 }
```

```

15         volume += qty;
16         tradeCount++;
17     }
18 }

```

Listing 7.1: Candle building (CandleAggregator.java)

Candles are upserted into the `stock_candles` table using `INSERT OR REPLACE` with a composite primary key of (`ticker`, `interval`, `open_time`).

7.4 Query

The `CandlesHandler` REST endpoint at `GET /api/v1/candles/{ticker}/{interval}/{limit}` returns sorted candle data for the chart:

```

1 public static List<Candle> getCandles(String ticker, String
   interval, int limit) {
2     String sql = "SELECT * FROM stock_candles WHERE ticker = ?
   AND interval = ? " +
3         "ORDER BY open_time DESC LIMIT ?";
4     // ... execute, reverse into ascending order ...
5 }

```

Listing 7.2: Candle query (CandleAggregator.java)

Chapter 8

Admin Commands

All stock market admin commands require the `market.admin` permission and are accessed under: `/market admin stock <subcommand>`.

8.1 Pause

Pauses the stock market engine. All order matching and stop-order evaluation stops immediately. New orders return an error with message “Stock market is currently paused by an admin.”

```
1 /market admin stock pause [reason]
```

Listing 8.1: Usage

Implementation (`StockMarketEngine.java`):

```
1 public static void pause() {  
2     paused = true;  
3     Bukkit.getLogger().warning("[Market] Stock market PAUSED.")  
4     ;  
5 }
```

Listing 8.2: Pause implementation

8.2 Resume

Resumes the stock market engine after a pause. Matching and stop-order evaluation resume on the next tick cycle.

```
1 /market admin stock resume
```

Listing 8.3: Usage

```
1 public static void resume() {  
2     paused = false;  
3     Bukkit.getLogger().info("[Market] Stock market RESUMED.");  
4 }
```

Listing 8.4: Resume implementation

8.3 Inject Test Trade

Injects a fake trade directly into the system without going through the order book. This sets the last price for the ticker and creates a trade record, which is broadcast to WebSocket subscribers. Useful for testing the full pipeline (database, REST API, WebSocket, frontend chart) without player interaction.

```
1 /market admin stock inject <ticker> <price> <quantity>
```

Listing 8.5: Usage

Example:

```
1 /market admin stock inject DIAMOND 150.25 10
```

Listing 8.6: Example

Implementation (StockMarketEngine.java):

```
1 public static Trade injectTrade(String ticker, double price,
2   int quantity) {
3     if (!loaded) return null;
4     ticker = ticker.toUpperCase();
5     lastPrices.put(ticker, price);
6     Trade trade = new Trade(
7       generateId("trd_"), ticker, price, quantity,
8       "00000000-0000-0000-0000-000000000000",
9       "00000000-0000-0000-0000-000000000000",
10      "injected", "injected", System.currentTimeMillis());
11   saveTrade(trade);
12   broadcastTrades(List.of(trade));
13   return trade;
14 }
```

Listing 8.7: Inject trade implementation

8.4 Status

Displays current stock market engine status including pause state, open order count, and 24-hour trade count.

```
1 /market admin stock status
```

Listing 8.8: Usage

Sample output:

```
1 --- Stock Market Status ---
2 Engine: RUNNING
3 Open orders: 12
4 Trades (24h): 45
```

Listing 8.9: Sample status output

8.5 Complete Subcommand Registration

The `StockCommand` nested command (`StockCommand.java`) registers all four subcommands:

```
1 public class StockCommand extends NestedCommand {
2     public StockCommand() {
3         registerSubCommand(new Pause());
4         registerSubCommand(new Resume());
5         registerSubCommand(new Inject());
6         registerSubCommand(new Status());
7     }
8     @Override
9     public String getRequiredPermission() { return "market.
admin"; }
10    @Override
11    public String getName() { return "stock"; }
12 }
```

Listing 8.10: `StockCommand` registration

Chapter 9

Frontend Application

9.1 Technology Stack

Build Vite 6 + TypeScript 5

UI React 18, React Router 6

Charting TradingView Lightweight Charts 4

Styling Tailwind CSS 3 (Catppuccin dark theme)

9.2 Project Structure

```
stock-website/
+-- src/
|   +-- api/
|   |   +-- client.ts      # fetch wrapper with cookie auth (credentials:include)
|   |   +-- auth.ts       # step1, step2, status, logout
|   |   +-- market.ts     # tickers, orderbook, trades, orders, portfolio
|   |   +-- websocket.ts   # WsClient with auto-reconnect + key transform
|   |   +-- transform.ts   # snake_case to camelCase normalizer
|   +-- store/
|   |   +-- auth.tsx       # AuthContext + AuthProvider
|   +-- pages/
|   |   +-- Login.tsx      # API key entry
|   |   +-- Verify2FA.tsx  # TOTP code entry
|   |   +-- Dashboard.tsx  # Main trading interface
|   |   +-- Portfolio.tsx  # Holdings with P&L
|   +-- components/
|   |   +-- Layout.tsx     # Header + router outlet
|   |   +-- Header.tsx     # Navigation + logout
|   |   +-- TickerList.tsx # Searchable ticker sidebar
|   |   +-- Chart.tsx      # Candlestick chart with interval selector
|   |   +-- OrderBook.tsx  # Bid/ask levels with depth bars
|   |   +-- TradeHistory.tsx# Recent trades feed
|   |   +-- OrderForm.tsx  # Buy/Sell with Market/Limit/Stop types
```



```
|   +-- OrderList.tsx    # Open orders table with cancel
+-- types/index.ts      # All TypeScript interfaces
```

9.3 Key Components

9.3.1 WebSocket Client (api/websocket.ts)

The `WsClient` class provides automatic reconnection with a 5-second backoff, typed event handling, and subscribe/unsubscribe:

```
1 export class WsClient {
2   private ws: WebSocket | null = null;
3   private handlers = new Map<string, Set<WsMessageHandler>>();
4   private reconnectTimer: ReturnType<typeof setTimeout> | null
      = null;
5
6   subscribe(channel: string) {
7     this.send({ type: 'subscribe', channel });
8   }
9
10  on(type: string, handler: WsMessageHandler) {
11    if (!this.handlers.has(type)) this.handlers.set(type, new
      Set());
12    this.handlers.get(type)!.add(handler);
13    return () => this.handlers.get(type)?.delete(handler);
14  }
15 }
```

Listing 9.1: WebSocket client class

9.3.2 Dashboard Layout

The Dashboard (`pages/Dashboard.tsx`) renders a multi-panel layout:

```
+-----+
| Header:  CN Market | Trade | Portfolio  | Player | Logout   |
+-----+-----+-----+-----+-----+
| Ticker   |      Candlestick Chart      | Order Book   |
| List     |      (Lightweight Charts)     | (bids / asks) |
| (sorted  |      + interval selector     +-----+
| by |%) |      | Trade History   |
+-----+-----+-----+-----+
| Order Form (Buy/Sell, Market/Limit/Stop Loss/Take Profit) |
+-----+-----+-----+-----+
| Open Orders table (ticker, side, type, price, qty, cancel) |
+-----+-----+-----+-----+
```

9.3.3 Candlestick Chart (components/Chart.tsx)

The chart uses TradingView's Lightweight Charts library. It fetches data from `GET /api/v1/candles/{ticker}/{interval}/200` and renders a candlestick series. An interval selector lets the user switch between 1m, 5m, 15m, 1h, and 1d granularities.

9.3.4 Order Form (components/OrderForm.tsx)

Supports all four order types with validation:

- Buy/Sell toggle button group
- Market, Limit, Stop Loss, Take Profit selector
- Conditional price input (required for Limit/Stop/Take Profit)
- Quantity input with minimum validation
- Real-time reference price display

9.4 Auth Flow

The React app uses `AuthContext` (stored in React state with `sessionStorage` persistence). The protected route wrapper (`ProtectedRoute`) redirects unauthenticated users to the login page:

```
1 function ProtectedRoute({ children }: { children: React.  
  ReactNode }) {  
2   const { sessionId, loading } = useAuth();  
3   if (loading) return <Loading />;  
4   if (!sessionId) return <Navigate to="/login" replace />;  
5   return <>{children}</>;  
6 }
```

Listing 9.2: Protected route logic

Chapter 10

Configuration

All stock market configuration lives in `plugins/Market/config.yml` under the `stock-api` key:

```
1 # Stock Market API -- embedded HTTP server + WebSocket
2 stock-api:
3   # Enable the embedded servers for stock market website
4   # integration
5   enabled: true
6   # Port the HTTP REST API server listens on
7   port: 8080
8   # Port the WebSocket real-time server listens on
9   ws-port: 8081
10  # Rate-limiting: max requests per window for REST API
11  rate-limit:
12    max-requests: 120
13    window-seconds: 60
```

Listing 10.1: `config.yml` stock-api section

10.1 Port Configuration

port (default 8080) The REST API server. The Vite dev server proxies `/api/*` to this port.

ws-port (default 8081) The WebSocket server. The frontend connects directly to `ws://hostname:8081`.

10.2 Rate Limiting

max-requests (default 120) Maximum number of REST API requests allowed within the window. Public endpoints are keyed by IP; authenticated endpoints are keyed by session UUID.

window-seconds (default 60) The sliding window duration in seconds.

Chapter 11

Building and Deployment

11.1 Prerequisites

- Java 21 JDK (for the Spigot plugin)
- Maven 3.8+ (for building the plugin)
- Node.js 20+ and npm (for the frontend)
- A Spigot/Paper 1.21.x Minecraft server

11.2 Building the Plugin

```
1 cd CN-Market-main
2 mvn clean package
```

Listing 11.1: Build plugin

The output JAR (`target/Market.jar`) is a shaded plugin containing all dependencies.

11.3 Running the Frontend

```
1 cd stock-website
2 npm install
3 npm run dev
```

Listing 11.2: Start frontend dev server

The Vite dev server runs on port 3000 and proxies `/api/*` to `localhost:8080` and `/ws` to `ws://localhost:8081`.

11.4 Production Build

```
1 cd stock-website
2 npm run build
```

Listing 11.3: Production frontend build

The output is in `stock-website/dist/` and can be served by any static file server.

11.5 Production Deployment (nginx)

A reverse proxy is required in production so that the REST API (~:8080), WebSocket (~:8081), and static frontend are all served on the same origin. A reference config is provided in `nginx-market.conf`:

```
1 # 1. Build the plugin JAR and frontend dist
2 mvn clean package
3 cd stock-website && npm run build && cd ..
4
5 # 2. Copy artifacts to server
6 cp target/Market.jar /srv/minecraft/plugins/
7 cp -r stock-website/dist/* /var/www/market/
8
9 # 3. Install nginx config
10 sudo cp nginx-market.conf /etc/nginx/sites-available/market
11 sudo ln -s /etc/nginx/sites-available/market /etc/nginx/sites-enabled/
12 sudo nginx -t && sudo systemctl reload nginx
```

Listing 11.4: Deploy with nginx

The nginx config handles:

- Static file serving from `/var/www/market` (with long-lived cache headers for `/assets/`)
- Reverse proxy for `/api/*` to `localhost:8080`
- WebSocket upgrade for `/ws` to `localhost:8081`
- SPA fallback (all routes serve `index.html`)
- Gzip compression and security headers

11.6 Docker Test Server

A `docker-compose.yml` is provided for local integration testing:

```
1 # Build the plugin first
2 mvn clean package
3
4 # Start PaperMC server with plugin mounted
5 docker compose up -d
6
7 # Watch server logs
```

```
8 docker compose logs -f mc-server
```

Listing 11.5: Start Docker test server

The Docker setup uses `itzg/minecraft-server` with Paper 1.21.x (version 26.1.2) and mounts `target/Market.jar` into the `plugins` directory.

11.7 Testing with Injected Trades

1. Start the Minecraft server with the plugin.
2. Generate an API key: `/market apikey generate`.
3. Inject test data: `/market admin stock inject DIAMOND 150.25 10`.
4. Open `http://localhost:3000`, paste the key, complete TOTP setup.
5. View the dashboard with injected trade data visible on the chart.

Chapter 12

Complete Source Listings

12.1 Key File Reference

File	Lines
stock/StockMarketEngine.java	1019
stock/EmbeddedServer.java	798
stock/WebSocketManager.java	434
stock/ApiKeyManager.java	328
stock/TwoFactorAuth.java	180
stock/CandleAggregator.java	195
stock/RateLimiter.java	65
stock/transform.ts (frontend)	15
subcommands/StockCommand.java	38
subcommands/stock/Pause.java	56
subcommands/stock/Resume.java	51
subcommands/stock/Inject.java	88
subcommands/stock/Status.java	78
subcommands/ApiKeyCommand.java	42
subcommands/apikey/Generate.java	60
subcommands/apikey/Renew.java	48
subcommands/apikey/Revoke.java	48
subcommands/apikey/Status.java	58
managers/DatabaseManager.java	957
Market.java	182
nginx-market.conf	52

12.2 Total Size

- Java source (stock package): ~3,250 lines
- Java source (all stock-related): ~4,600 lines
- TypeScript/React source: ~1,200 lines
- Total: ~5,800 lines of new code